

Chaos Engineering Automated Canary Deployer

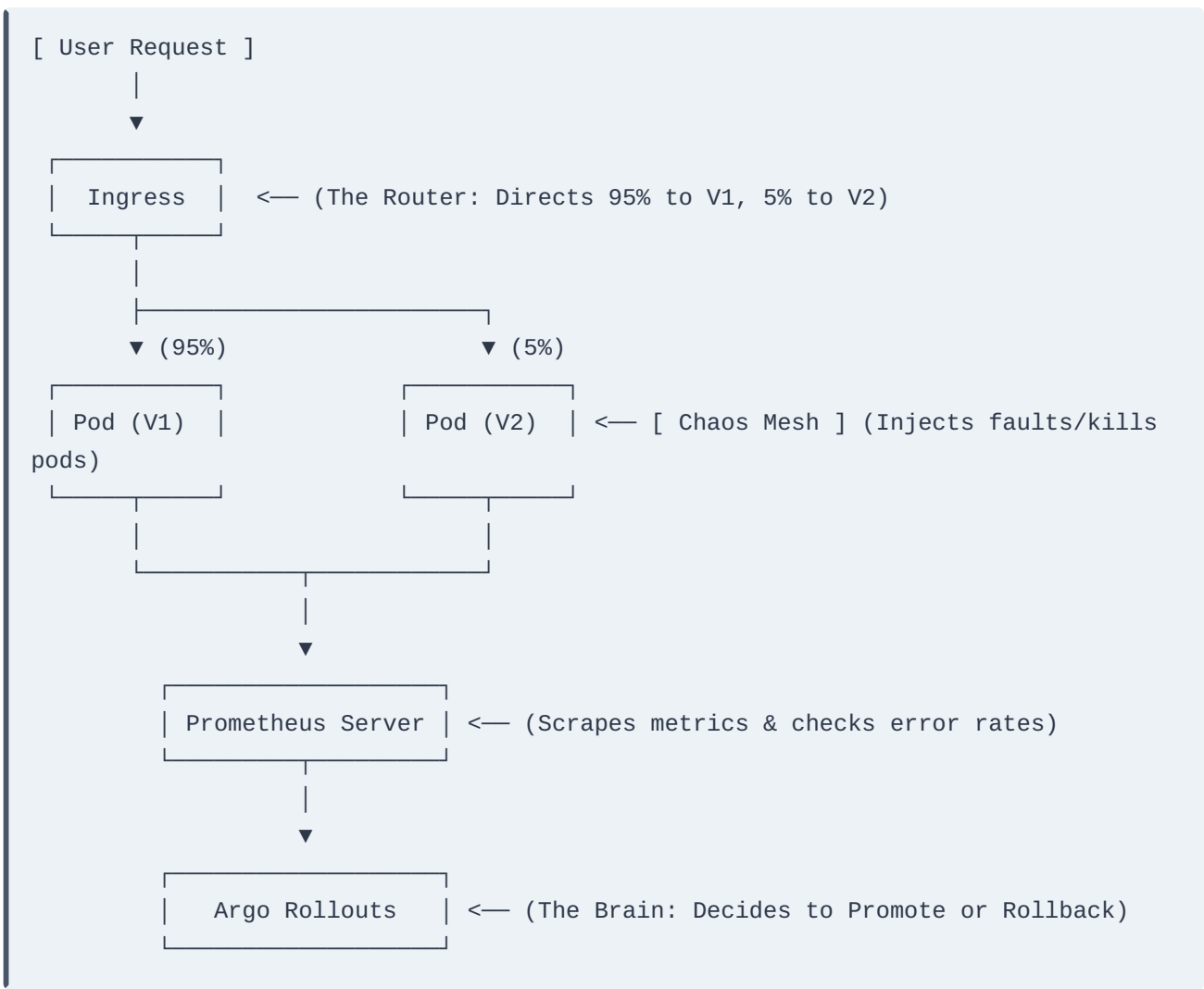
Project Study Guide & Master Roadmap

1. The Architecture Concept

The objective of this project is to build an automated continuous delivery pipeline that mitigates deployment risks through progressive traffic shifting (Canary Deployments) while simultaneously validating system resilience by actively injecting operational failures (Chaos Engineering).

System Architecture Flow

When a request enters the infrastructure, it follows this topological pathway:



2. Structured Learning Roadmap

To successfully implement this project on a local laptop environment, study the foundational components in the following chronological phases:

Phase 1: Core Kubernetes Foundations

Understand how containerized applications are orchestrated differently than standalone Docker Compose environments.

- **Pods:** The smallest deployable units in Kubernetes, wrapping one or more Docker containers with shared storage and network namespaces.
- **Deployments:** Declarative state managers that handle replica scaling, health checks, and basic rolling updates.
- **Services (ClusterIP & NodePort):** Internal abstraction layers that expose network endpoints natively to proxy traffic between pods or out to the host laptop.

Phase 2: Advanced Traffic Management

Shift away from static configurations to dynamic network routing mechanisms.

- **Ingress Controllers:** Cluster edge application proxies (e.g., NGINX Ingress) managing reverse proxy setups and routing rules via hostnames/paths.
- **Argo Rollouts (Custom Resource Definitions):** An alternative controller replacing default deployments to allow precise, progressive step-based traffic splitting.

Phase 3: Automated Observability

Equip the infrastructure with the telemetry required to evaluate deployment health metrics.

- **Prometheus Metrics:** Instrumenting web applications to expose time-series analytical counters (e.g., HTTP request totals, 5xx error metrics) at a `/metrics` target endpoint.
- **PromQL:** The querying syntax utilized to process streaming application telemetry data to determine real-time error percentage rates.

Phase 4: Chaos Engineering Orchestration

Introduce automated, programmatic system fault injection paradigms.

- **Chaos Mesh / LitmusChaos:** Custom testing frameworks capable of introducing structural infrastructure stress (e.g., `PodKill` disruptions, network delays, CPU resource hogging) to validate cluster self-healing capabilities.

3. The Step-by-Step Execution Plan

Implementation Strategy: Do not build all components concurrently. Complete each task completely before moving to the next layer of complexity.

1. **Develop a Sample Microservice:** Write a minimal web service in Python (FastAPI/Flask) or Node.js. Implement an explicit `/health` JSON endpoint and package it into a Docker image.
2. **Configure the Sandbox Environment:** Initialize a lightweight local cluster engine (Minikube or Kind) directly onto the Windows host machine utilizing the local Docker driver. Write standard Kubernetes deployment manifests to host and access your sample application locally.
3. **Implement Progressive Delivery:** Deploy the Argo Rollouts controller and active Ingress controller into your sandbox. Transition your application configurations into an explicit Rollout manifest that stops at a 5% canary traffic split. Verify functionality locally using CLI utilities.
4. **Establish Metrics Integration:** Expose operational application telemetry, deploy a Prometheus instance within the cluster, and formulate an Argo AnalysisTemplate query. Configure the automated pipeline to execute a fast rollback to the previous deployment version if error limits exceed a 1% threshold.
5. **Execute Resiliency Verification:** Set up Chaos Mesh targeting your canary instances. Trigger a new deployment upgrade while simultaneously orchestrating automated runtime failures. Observe your cluster to verify that it autonomously remediates the errors or reverts safely without human intervention.